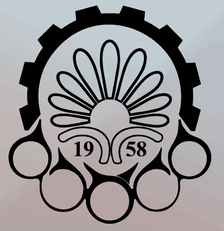
Learning by Experiencing: Deep Reinforcement Learning

Computation Intelligence and its Application in Mechatronics

Spring 2025



Amirkabir University of Technology
(Tehran Polytechnic)

A Quick View

- Teach an agent to make decisions that maximize long-term reward.
- The agent **learns from experience** by interacting with the environment: it observes a state, takes an action, receives a reward, and transitions to a new state.
- Through **trial and error**, it improves its strategy (policy) over time by reinforcing actions that lead to higher rewards.

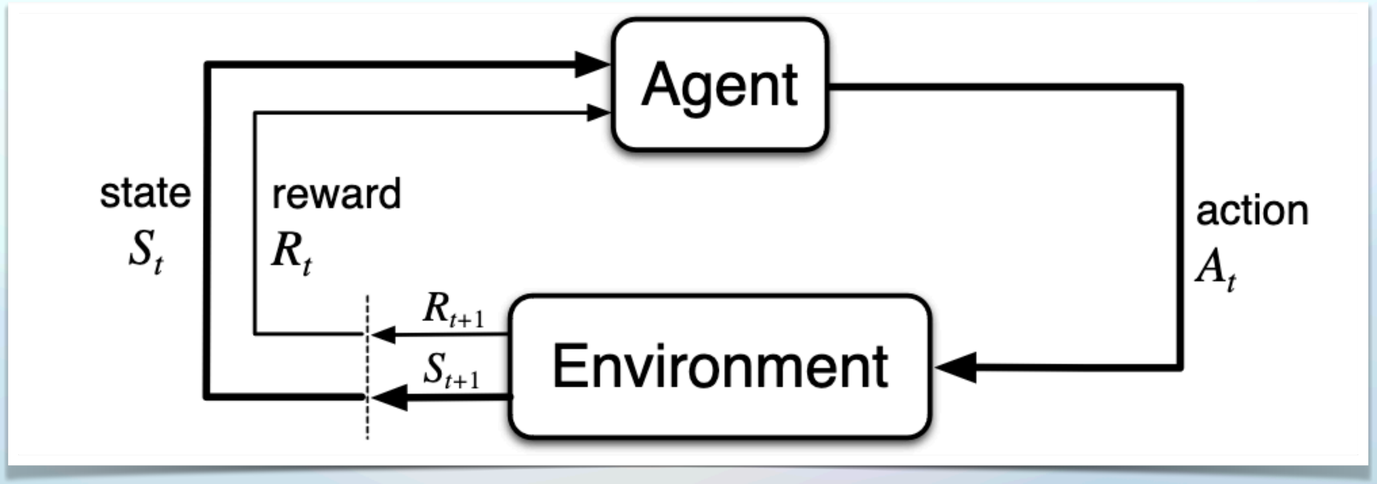


Image from: https://medium.com/quantumblack/taking-the-best-intervention-with-reinforcement-learning-1b49c6253f05

RL Glossary

- **Agent:** The software program that learns to make intelligent decisions.
- **Environment:** The world of the agent.
- **State:** A position or a moment in the environment that the agent can be in.
- **Action:** The agent interacts with the environment by performing an action and moving from one state to another.
- **Reward:** A numerical value that the agent receives based on its action.
- **Policy** $\pi(a \mid s)$: A policy tells the agent what action to take in each state.
- **Episode**: The agent-environment interaction from the initial state to the terminal state is called an episode.

RL Glossary

- **Episodic and continuous tasks**: An RL task is called an episodic task if it has a terminal state, and it is called a continuous task if it does not have a terminal state.

- **Return**: Return is the sum of rewards the agent receives in an episode.

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \ldots = \sum_{k=0}^{\infty} r_{t+k+1}$$

- **Discounted Return**: The sum of future rewards, where each reward is multiplied by a discount factor, prioritizing immediate rewards.

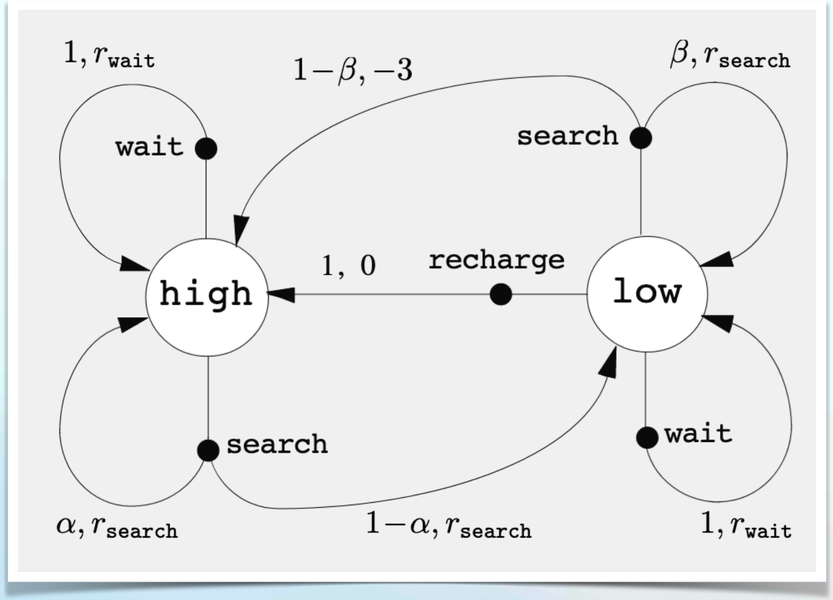
$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- **Value function**: The expected return an agent would get starting from state s following policy π.

- **Q function**: The expected return an agent would obtain starting from state s and performing action a following policy π.

The Environment Formulation

Markov Decision Process (MDP)



s	a	s'	$p(s' \mid s, a)$	$r(s, a, s')$
high	search	high	α	$r_{\texttt{search}}$
high	search	low	$1-\alpha$	$r_{\texttt{search}}$
low	search	high	$1-\beta$	-3
low	search	low	β	$r_{\texttt{search}}$
high	wait	high	1	$r_{\texttt{wait}}$
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	$r_{\texttt{wait}}$
low	recharge	high	1	0
low	recharge	low	0	-

Image from: Sutton, R.S. and Barto, A.G., 1998. *Reinforcement learning: An introduction* (Vol. 1, No. 1, pp. 9-11). Cambridge: MIT press.

Value Functions

State-Value function:

$$v_\pi(s) \doteq \mathbb{E}_\pi\left[G_t \mid S_t = s\right] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s\right] = \sum_a \pi(a \mid s) \sum_{s',r} p\left(s', r \mid s, a\right)\left[r + \gamma v_\pi(s')\right]$$

Action-Value function:

$$q_\pi(s,a) \doteq \mathbb{E}_\pi\left[G_t \mid S_t = s, A_t = a\right] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right]$$

Bellman Optimality Equations

Optimal state-value function:

$$V_*(s) = \max_a \sum_{s'} P(s' \mid s, a)\left[R(s, a, s') + \gamma V_*(s')\right]$$

Optimal action-value function:

$$Q_*(s, a) = \sum_{s'} P(s' \mid s, a)\left[R(s, a, s') + \gamma \max_{a'} Q_*(s', a')\right]$$

$V_*(s)$: Optimal value of state s

$Q_*(s, a)$: Optimal value of taking action a in state s

$P(s' \mid s, a)$: Transition probability

$R(s, a, s')$: Reward received after transitioning to state s'

γ: Discount factor

Main Approaches

Policy-Based Methods: We learn a policy function directly.

Value-based methods: Instead of learning a policy function, we learn a value function that maps a state to the expected value of being at that state.

The Deep Q-Learning Algorithm

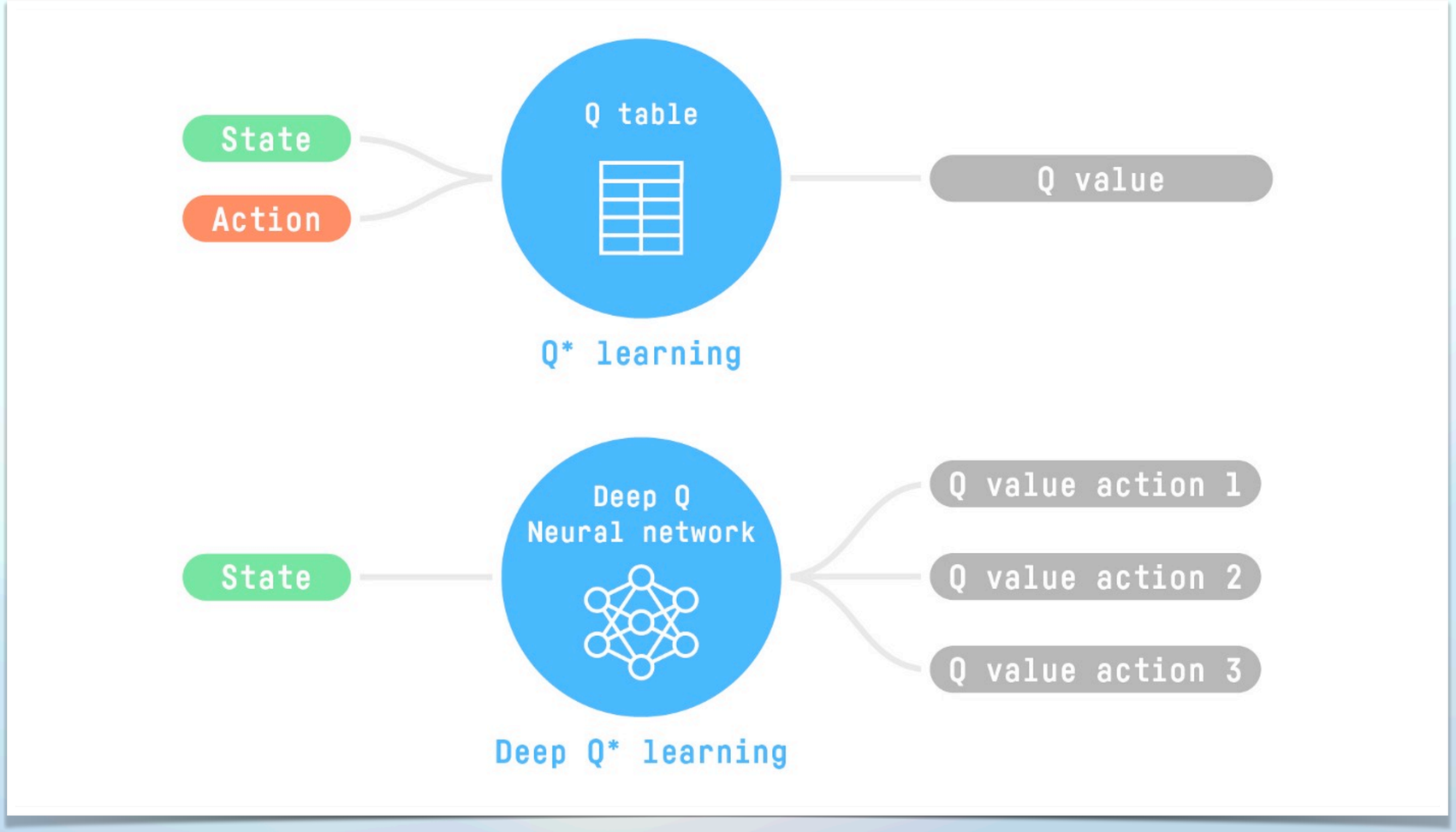


Image from: https://huggingface.co/learn/deep-rl-course/unit3/from-q-to-dqn

The Deep Q-Learning Algorithm

Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih **Koray Kavukcuoglu** **David Silver** **Alex Graves** **Ioannis Antonoglou**

Daan Wierstra **Martin Riedmiller**

DeepMind Technologies

{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

Abstract

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural network, trained with a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards. We apply our method to seven Atari 2600 games from the Arcade Learning Environment, with no adjustment of the architecture or learning algorithm. We find that it outperforms all previous approaches on six of the games and surpasses a human expert on three of them.

The Deep Q-Learning Algorithm

Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

New Q-value estimation

Former Q-value estimation

Learning Rate

Immediate Reward

Discounted Estimate optimal Q-value of next state

Former Q-value estimation

TD Target

TD Error

Image from: https://huggingface.co/learn/deep-rl-course/unit3/from-q-to-dqn

The Deep Q-Learning Algorithm

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

Image from: Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D. and Riedmiller, M., 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*.

Policy Gradient Methods

In policy-gradient methods, a subclass of policy-based methods, we search directly for the optimal policy. However, we optimize the policy network parameter directly by performing the gradient ascent on the performance of the objective function.

Policy Gradient Methods

Advantages

- The simplicity of integration
- Policy-gradient methods can learn a stochastic policy
- Policy-gradient methods are more effective in high-dimensional action spaces and continuous actions spaces
- Policy-gradient methods have better convergence properties

Disadvantages

- Frequently, policy-gradient methods converge to a local maximum instead of a global optimum.
- Policy-gradient goes slower, step by step: it can take longer to train (inefficient).

Policy Gradient Methods

Algorithm 1 Vanilla Policy Gradient Algorithm

1: Input: initial policy parameters θ_0, initial value function parameters ϕ_0
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
4: Compute rewards-to-go $\hat{R}_t$.
5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.
6: Estimate policy gradient as

$$\hat{g}_k = \frac{1}{|\mathcal{D}_k|} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)|_{\theta_k} \hat{A}_t.$$

7: Compute policy update, either using standard gradient ascent,

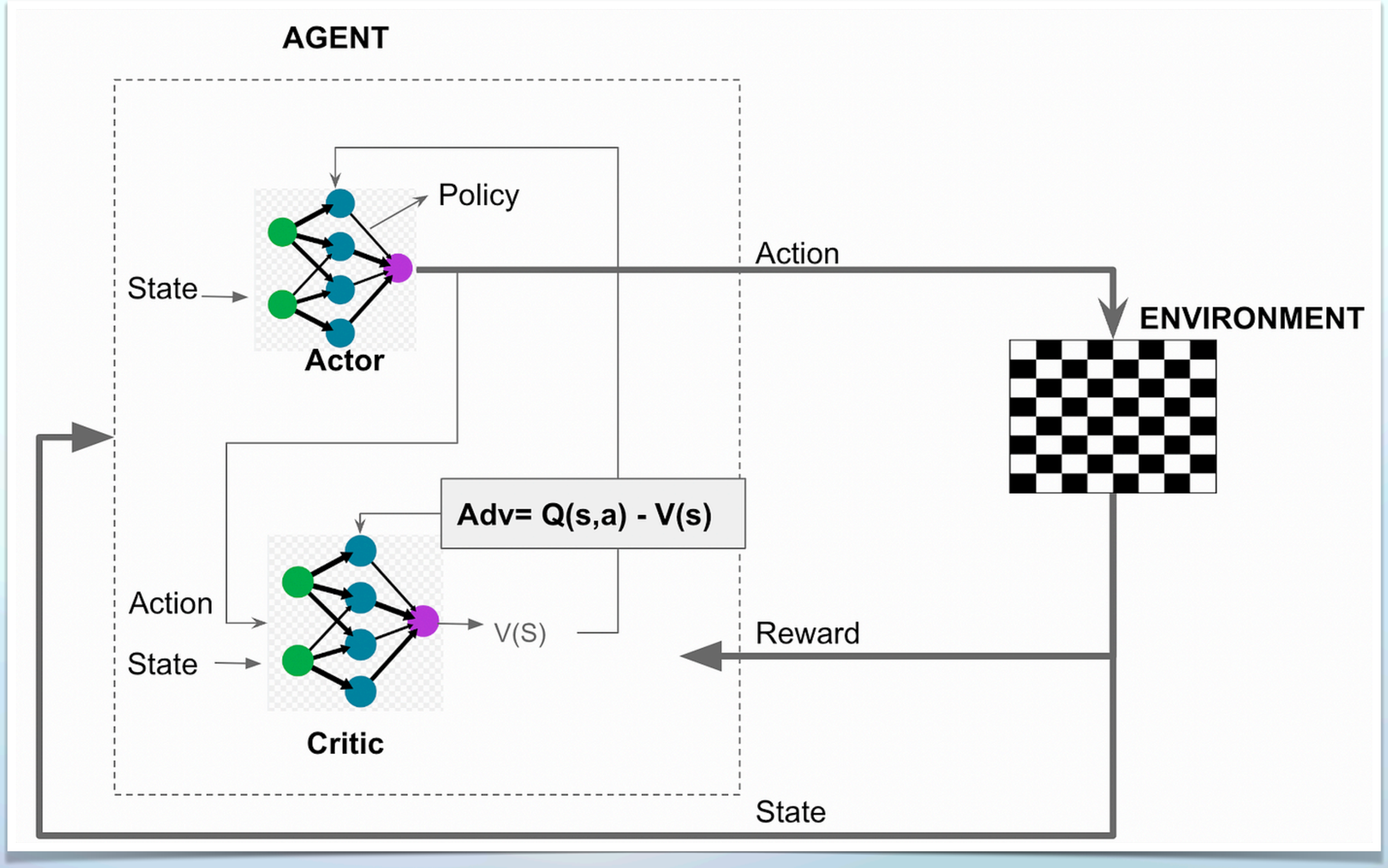
$$\theta_{k+1} = \theta_k + \alpha_k \hat{g}_k,$$

or via another gradient ascent algorithm like Adam.
8: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_\phi \frac{1}{|\mathcal{D}_k| T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.
9: **end for**

Actor-Critic Methods



Actor-Critic Methods

Actor (Policy): The actor makes decisions by selecting actions based on the current policy. Its responsibility lies in exploring the action space to maximize expected cumulative rewards. By continuously refining the policy, the actor adapts to the dynamic nature of the environment.

Critic (Value Function): The critic evaluates the actions taken by the actor. It estimates the value or quality of these actions by providing feedback on their performance. The critic's role is pivotal in guiding the actor towards actions that lead to higher expected returns, contributing to the overall improvement of the learning process.

Actor-Critic Methods

Advantage Function (A(s,a))

Measures the advantage of taking action a in state s over the expected value of the state under the current policy:

$$A(s, a) = Q(s, a) - V(s)$$

Actor-Critic Methods

Actor Critic Algorithm Objective Function - Policy Gradient (Actor)

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=0}^{N} \nabla_\theta \log \pi_\theta(a_i|s_i) \cdot A(s_i, a_i)$$

J(θ) represents the expected return under the policy parameterized by θ

$\pi_\theta(a \mid s)$ is the policy function

N is the number of sampled experiences.

A(s,a) is the advantage function representing the advantage of taking action a in state s.

i represents the index of the sample

Actor-Critic Methods

Actor Critic Algorithm Objective Function - Value Function Update (Critic)

$$\nabla_w J(w) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_w (V_w(s_i) - Q_w(s_i, a_i))^2$$

$\nabla_w(J(w))$ is the gradient of the loss function with respect to the critic's parameters w.

N is number of samples

$V_w(s_i)$ is the critic's estimate of value of state s with parameter w

$Q_w(s_i, a_i)$ is the critic's estimate of the action-value of taking action a

i represents the index of the sample

Actor-Critic Methods

Actor Critic Algorithm Objective Function - Update Rules

Actor:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t)$$

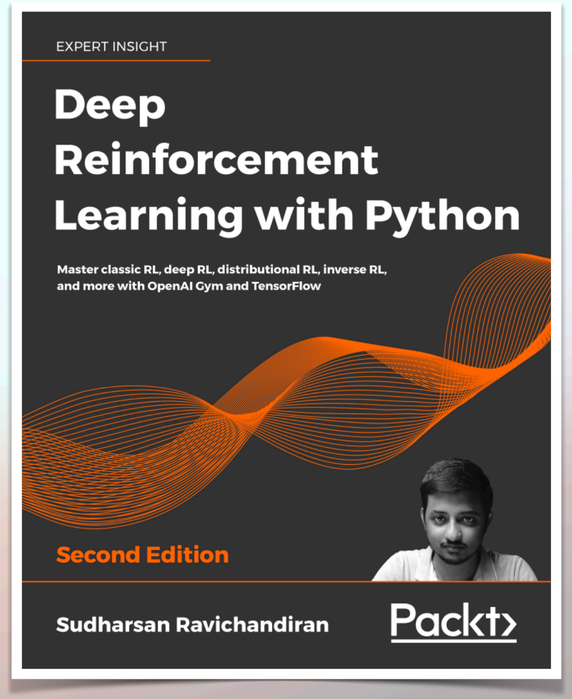
Critic:

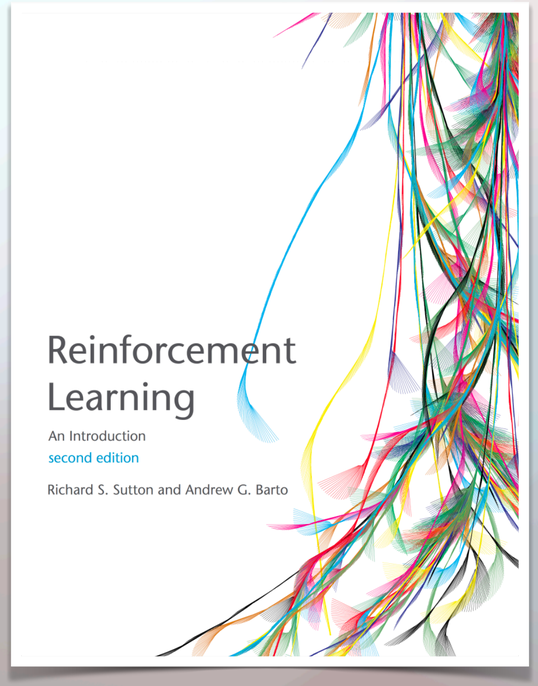
$$w_t = w_t - \beta \nabla_w J(w_t)$$

Some Good References

OpenAI Gym Documentation

Make your own custom environment in Gym





Some Good References

Hugging face Deep RL course

Some Good References

Repository Link github.com/amirhosseinh77/Synapse-RL

RL Algorithm	Description
`Deep Q Learning`	Discrete
`Policy Gradient`	Discrete
`Actor Critic (A2C)`	Discrete
`Deep Deterministic Policy Gradient (DDGP)`	Continuous
`Soft Actor Critic (SAC)`	Continuous
`Proximal Policy Optimization (PPO)`	-